Donghyung Lee

DevOps Engineer | Career Description

GitHub: github.com/somaz94 Blog(EN): somaz.blog Blog(KR): somaz.tistory.com Portfolio: somaz.blog/portfolio

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- **Cost optimization** – 20% via hybrid transition, 50% via GCP migration (Fargate → GKE node pools + MSP discount)
- **Greenfield EKS and IaC** – sole build and launch of a new game's EKS (platform 100% declared in Git, excluding IAM), new components 100% IaC with Terraform and Ansible (excluding IAM)
- **GPU infrastructure for AI workloads** – GCP A100 / L4 GPU VMs in Terraform (NerdyStar), an in-house LLM built on an on-prem GPU (Phantom)

CI/CD Pipeline & GitOps

Improving development productivity through GitOps-based deployment automation

- **Pipeline design & hardening** – multi-cluster deployment automation on GitHub Actions, GitLab CI, Jenkins and ArgoCD, 67% shorter deploys
- **CI toolchain standardization** – layered shared images and an automated version cycle, provisioning 5min → ~10s (~97%)

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- **Logs, metrics and traces** – logging taken from ELK to EFK to the ECK Operator, kube-prometheus-stack alerting, and OpenTelemetry + Tempo linking metrics, traces and logs

Automation & Team Enablement

Operations automation tooling with Python and Bash, plus training and standardized materials that let partner engineers run things themselves

- **Engineering enablement** – designed and ran a 6-month hands-on Kubernetes course for the DP (Delivery Partner) field engineers (avg 10 attendees, 4h/session)
- **Operations automation tooling** – infrastructure tooling ported to Python, chart-upgrade MRs opened automatically, and the Kubespray upgrade and its checks scripted

Kubernetes Platform Modernization & Open-Source Contribution

Gateway API (NGF) migration, public charts released as open source, and upstream contribution

- **Open source, published and upstream** – Helm charts, CRD controllers, GitHub Actions and Ansible collections released (ArtifactHub, GitHub Marketplace, Ansible Galaxy); Gateway API HTTPRoute support in the apache/airflow, OpenTelemetry and prometheus-community charts; contributions to kubernetes-sigs (Kueue, KWOK), Prometheus Operator, k3s and Gitea
- **NGINX Gateway Fabric upstream contribution** – policy status conditions, a refactor of 11 controller functions and a BackendTLSPolicy panic fix, credited as a community contributor two releases in a row, in the NGF 2.7 and NGF 2.8 release announcements

Infrastructure Security & IAM

Centralized in-house authentication and secret governance via Keycloak Operator SSO and Vaultwarden

- **Authentication and secrets** – SSO on the Keycloak Operator (ArgoCD, Harbor), plaintext AWS production DB passwords → External Secrets Operator + AWS Secrets Manager, pull secret → Sealed Secrets

Professional Experience

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

The sole DevOps engineer for a game studio's infrastructure: designed and run an on-premises + AWS hybrid architecture that cut monthly infrastructure cost 20%, with second-line support for the live game and infrastructure for new titles.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution: hybrid architecture design & build 100% (solo) / second-line technical support and operations for the live game in collaboration with the external publisher's ops team 2024.10 ~ Present (In Operation)

Problem

Everything ran on AWS, so even non-production environments with steady traffic held cloud resources around the clock; the estate needed cost optimization and environment separation.

Approach

- **Hybrid architecture design** – analyzed cost and usage patterns per workload, then moved dev·qa on-premises and kept review·prod, the environments that need scale and high availability, on AWS (EKS, ALB)
- **Traffic and registry separation** – traffic separated at the application layer, and the registry split (dev·qa = Harbor, review·prod = ECR)
- **IaC-based operational standardization** – introduced Terraform and Ansible across both on-premises and AWS, reaching 100% IaC for new components (excluding IAM) and establishing one configuration-management standard across heterogeneous infrastructure

Results

- Cut monthly infrastructure cost 20% by re-placing workloads on the analysis, with the saving reinvested into new project infrastructure

Project 2: Game Application CI/CD Pipeline & Shared Toolchain Standardization (App GitOps)

Contribution 100% (Sole design and build across the pipeline and CI standardization) 2024.12 ~ Present (In Operation)

Automated game application delivery with GitOps and standardized the CI toolchain images and versioning every in-house repository shares.

2-1. Game Application Deployment CI/CD Pipeline (App GitOps, 2024.12 ~ Present)

Problem

Developers built and deployed to the servers by hand — about 30 minutes per deploy, with human error causing frequent incidents.

Approach

- **GitOps continuous delivery pipeline** – GitLab CI wired to ArgoCD so a Git push drives build, test and deploy, with drift from Git self-healed through an ArgoCD app-of-apps structure; all four environments — dev, qa, review and prod — deploy through that one pipeline
- **Least-privilege image build environment** – builds moved from Docker-in-Docker, which needs privileged mode, to daemonless Kaniko, lowering the risk that a compromised build job takes over the host

Results

- 67% shorter deployment time (30min → 10min) — the manual steps unified into the pipeline, eliminating human error as a root cause of deployment incidents
- ~78% shorter source fetch in the data deployment (~139s → ~30s) — a shallow + blobless fetch takes only the latest commit and the files needed
- An average of 100 deploys a week per core service repository, and a design-data (MasterData) change automatically triggers the deployment pipeline of every downstream repository that consumes it

2-2. Shared Toolchain Image Layers & Automated Version Cycle (2025.09 ~ Present)

Problem

Every CI job re-installed the required tools (helm · helmfile · kubectl · crane · aws-cli), so build-environment provisioning averaged 5 minutes, and tool versions were scattered per repository, so a new version meant editing all of them.

Approach

- **Two-layer shared tool images**: external registry images mirrored into the internal Harbor with multi-arch (amd64 · arm64) manifests intact (Layer 1), and six shared images with the tools pre-installed on top (Layer 2), so a repository only pins an image tag
- **Shared CI templates and a distributed cache**: the CI jobs each repository copied became a shared template referenced through `include`, and with no S3-compatible store on-premises, an S3 cache mode for GitLab Runner was added to the open-source static file server I maintain (adopted by 10 repositories)
- **Automated version cycle**: one run takes new upstream versions through to the mirror and rebuild MRs, a central version-variable change propagates to every downstream repository, and merges are approved by a person; after helmfile 1.5.1 demanded helm 3.18.6 against the pinned 3.16.4 and stopped the build, only patches within the current minor are bumped automatically

Results

- **CI provisioning time cut ~97% (5min → ~10s)** — the pre-built standard images removed the build delay, with 100% tool-version consistency across the internal repositories
- Upgrade targets split into risk tiers (stateless, platform, data) generate child pipelines from one list, with automated MRs merged and applied to clusters only on a person's approval and a pre-flight check on the data tier

Project 3: Central Logging & Observability Platform (Collection → Integrity/HA → Product Analytics)

Contribution 100% (Sole design across the full lifecycle — collection · integrity · product analytics) 2024.11 ~ Present

Took central logging through three stages and extended it into retention analytics.

3-1. Centralized Logging System (ELK → EFK → ECK Operator 3-stage Enhancement, 2024.11 ~ Present)

Problem

Server and container logs were scattered, so a pod's logs vanished when it was recreated, and an incident meant connecting to nodes and pods one by one to find them.

Approach

- **A lighter EFK tier and the ECK Operator**: to cut the JVM-based Logstash's memory load, collection moved to a Fluent-bit DaemonSet and parsing to Fluentd, and with the official Helm chart stalled for over two years, the ECK Operator's CRDs now manage the lifecycle declaratively
- **Extension to the cloud (EKS) with keyless automation**: the same stack was templated onto EKS, with IRSA on ILM and the S3 snapshot backup path so neither needs a static credential

Results

- Logs survive a pod being recreated or terminated, and logs once checked pod by pod are traced in time order in one Kibana view
- TLS certificate renewal, account provisioning and node replacement automated through CRD declarations, and the Elastic Stack major upgrade completed (8.x → 9.x)

3-2. Fluent-bit Log Collector Data Integrity & HA Hardening (2026.03 ~ Present)

An SQLite offset DB on node-local storage keeps collector restarts at zero duplicate ingestion, and a stalled collector now alerts.
Traced in the plugin code why Fluentd dropped failed sends without retry during an Elasticsearch rolling restart, and made each batch retry as a whole.

3-3. Game User Retention Analytics Platform (EFK Logs → Product Analytics, 2026.06 ~ Present)

An Elasticsearch Transform (every 5 minutes) pivots in-game events to one row per user, so Kibana alone serves D+1~D+30 retention and cohort analysis with no SaaS.

Project 4: Developer Productivity Tools & a Self-Service Ops Console

Contribution 100% (Full system design & development) 2025.02 ~ Present

Built six internal tools that take repetitive work off the dev team and brought them together in a self-service console.

4-1. APK Delivery Automation: Slack Bot and Static File Server (2025.02 ~ Present)

A bot posting a download QR to Slack when a Jenkins APK build finishes cut QA delivery time 90%, and the old file server was replaced by a Go static file server I built and open-sourced.

4-2. GitLab-Google Drive Document Automation System (2 weeks, 2025.06)

Design documents sync from GitLab to Google Drive on push, cutting upload time from 10min to 2min.

4-3. Internal LLM Service Deployment (4 weeks, 2025.11)

Built an in-house LLM service in four weeks on an on-prem GPU server with Ollama and Open WebUI, with the coding model resident in GPU memory, which served the dev team as a shared tool.

4-4. Git Repository Mirroring Tool + Retry API Follow-up (4-week initial build, 2026.02 ~ Present)

Built and open-sourced git-bridge, a Go mirroring tool that ends manual syncing between CodeCommit, GitLab and GitHub; with retries, Slack alerts and a shutdown drain (measured median 2.9s), it runs as the standing sync path between the new and old in-house GitLab.

4-5. Internal Self-Service Operations Console Platform (2026.07 ~ Present)

Problem

Creating and deleting accounts depended on scripts only the infrastructure engineer could run, so every request went through one person, and permissions across 10 scattered account systems were hard to audit.

Approach

- **A single OIDC gateway and one portal**: 7 applications — the internal web tools plus 5 operations services (accounts · Keycloak · Harbor · Elasticsearch · VPN) — sit behind one central OIDC gateway, with the 10 account systems wired in and every account change recording who made it

Troubleshooting

- **Downtime from a race between the CI build and the GitOps sync**: ArgoCD synced a new tag before its image had finished building, causing `ImagePullBackOff`, and the services on RWO volumes and so the `Recreate` strategy (6 of 8) went fully down.
  A PreSync hook that actually pulls the image before the sync fixed it (on the Harbor ops service, a ~180s outage became a ~20s normal swap)

Results

- **The operational bottleneck removed through self-service** – designed and built solo over 6 weeks, so account and permission requests that went through one engineer are handled by each team from the console, within its own scope

---

Project 5: Kubernetes Cluster & Internal Systems Operations Enhancement

Contribution 100% (Solo design, with the standard procedures captured as automation scripts the team reuses) 2025.12 ~ Present

Overhauled the operation of the clusters and the internal systems.

5-1. Monitoring & Alerting Enhancement (2025.12 ~ Present)

Severity-based alerting on kube-prometheus-stack with lower alerts inhibited under a firing higher one stops duplicate notifications, and after Prometheus died unnoticed when its CRD was deleted, a watchdog on a bastion outside the cluster checks the monitoring stack every 5 minutes.

5-2. K8s Upgrade Automation & Helm Chart Management Framework (2025.12 ~ Present)

Scripted the Kubespray upgrade and post-upgrade checks; in the 2026.11 on-premises upgrade from 1.34 to 1.36, tuning only the DB node's drain grace cut the DB outage from about 7 minutes to about 4, and backup verification that had been passing without checking anything under etcd 3.6 was found and fixed.

5-3. Ingress-nginx → NGINX Gateway Fabric (NGF) Migration (2026.04)

Consolidated the ingress-nginx instances onto a single NGF control plane, keeping the MetalLB IPs so each class cut over in 30-60s with no DNS or firewall change.
Later found NGF accepting HTTPRoute timeouts yet cutting requests at its 60-second default, moved them to ProxySettingsPolicy and blocked them in the shared chart.

5-4. Storage Performance Optimization: etcd / DB SSD Migration and Build I/O Isolation (2 weeks, 2026.04 ~ 2026.05)

GitLab Runner disk I/O during builds was slowing HDD-backed etcd and the game DB; moving the control plane and DB worker to SSD and builds onto a VM on separate hardware brought game DB COMMIT latency from 9~14s back to milliseconds, with zero slowdowns during builds.

5-5. Kubernetes Infrastructure Deployment Automation & Multi-cluster GitOps (ArgoCD) Migration (2026.03 ~ Present)

Problem

The Shell/awk infrastructure monorepo could not be test-automated, and deployment was a push from CI runners (`helmfile apply`), so drift from Git went undetected and the setup did not scale to multiple clusters.

Approach

- **Python tooling with wave-based deploys and upgrades**: refactored onto dependency-free Python modules with unit tests, with a six-stage wave order and a pipeline that opens an MR carrying the values diff and compatibility checks for each new chart version
- **Moved to ArgoCD pull-GitOps**: ApplicationSets automate deployment on the on-prem and AWS clusters, and existing Helm releases were adopted without recreating a resource or an interruption
- **The infrastructure repo split into three by role**: one test, 'is it needed to bring up or recover ArgoCD?', separates the bootstrap Helmfile components, the components ArgoCD deploys, and host and VM operations; while the Git source was swapped, a temporary delete-protection option kept the move disruption-free
- **GitOps governance and safety rails**: cascading prune blocked at the top layer and an AppProject whitelist limiting the blast radius; a name wildcard that let the test project deploy into the production cluster was found, destinations now match by server address only, and a validator check prevents a regression

Results

- Platform releases are deployed by each on-prem and AWS cluster's own ArgoCD from the same Git repositories, and nobody tracks new versions or deployment order by hand anymore

5-6. Backup & Disaster-Recovery with Backup Verification (2026.08 ~ 2026.09)

Built backup and recovery across 8 internal systems and 2 NAS paths, and verifying that each file was written completely surfaced a job that exited 0 on a truncated DB dump, a backup that would have failed at restore.

5-7. Splitting the In-House GitLab in Two, and the Legacy Major Upgrade: a 21-Hop Path (new instance 2024.10 / upgrade 2026.08 ~ 2026.09)

Built a new bare-metal GitLab right after joining, kept the old one synced by git-bridge, my own tool, and upgraded the stalled old instance along the official Upgrade Path — 21 hops over 6 days, about 6.5 minutes of downtime on the first hop and 3-4 minutes on each after, and the nightly sync's transfer cut from 123G to 39G (–68%).

Project 6: Infrastructure Security & Governance

Contribution 100% (Design, deployment, SSO integration) 2026.04 ~ Present

Brought sign-in, secrets and remote access under one standard.

6-1. Central Authentication (SSO/OIDC) on Keycloak Operator (2026.04 ~ Present)

Consolidated the GitLab OIDC logins wired separately into Harbor and ArgoCD onto a central IdP on the Keycloak Operator, with GitLab groups flowing through Keycloak into ArgoCD roles so permissions live in one place

6-2. Secret Governance: Vaultwarden · GitOps Secrets · Token Audit (2026.04 ~ Present)

Moved scattered shared passwords onto Vaultwarden behind SSO, and moved the AWS production DB credentials once committed to Git in plaintext onto External Secrets Operator and AWS Secrets Manager, and the Harbor pull secret onto Sealed Secrets.
Tokens and mirror health are audited weekly.

6-3. Secure, Redundant Remote Access to the Internal Network (OpenVPN PKI, 2026.07 / HA 2026.10)

Built a PKI-based VPN gateway for 50 concurrent connections with per-user certificate issuing and revocation scripted.
The VPN and the AWS WireGuard tunnels, which sat on one server, later moved onto a keepalived active-standby pair, (3-4 seconds to detect a failure), and a failover drill measured an AWS path gap of 1.5 seconds or less.

Project 7: AWS EKS Production Game-Launch Platform (New Externally-Published Title)

Contribution 100% (solo design, build & operation) 2026.06 ~ 2026.11 (then moved to Project 8)

Solely built the EKS production environment for a new externally-published game, added tracing, canary delivery and CDN security for the soft launch, then SLI/SLOs.

7-1. Greenfield EKS Production Infrastructure & Live-traffic Tuning (2026.06 ~ 2026.11)

Problem

Under the rule that every product runs in its own AWS account, the new externally-published game had to launch commercially overseas from a new one, so a highly available EKS production environment was built from scratch in that account rather than joining the existing clusters.

Approach

- **A GitOps-managed platform, nodes split by workload**: the bootstrap tier (Karpenter, LBC, ArgoCD) goes through helmfile and every other platform component through ArgoCD, and all Karpenter pools run on ARM — the game services on on-demand so no node is reclaimed mid-session, CI runners spread across several spot instance families
- **Networking and authentication automated**: AWS Load Balancer Controller, ExternalDNS and a wildcard ACM certificate turn a declared Ingress into ALB rules, DNS and HTTPS, and every ServiceAccount runs on Pod Identity / IRSA with secrets pulled through External Secrets Operator, removing static IAM keys

Troubleshooting

- **Idle nodes never reclaimed**: a 2 GiB memory request against a measured 101 MiB came down to 512 MiB, so nodes once lopsided at 92% and 35% fill evenly and consolidation works again
- **Pods packed on node replacement, then a consolidation loop**: an AMI rotation put a service's two pods on one new node and left it Degraded for 17 seconds, fixed with a node spread constraint counted per deployment revision.

Applying it to a second service made underutilization consolidation relaunch a node every ~7.5 minutes, 16 times over, so consolidation went back to empty nodes only rather than keep evicting live pods

Results

- A production EKS environment built from scratch in the new game's own AWS account, with 100% of the platform manifests declared in Git (excluding IAM)
- Removed the bastion SSH key (deploys moved to Jenkins → S3 → CloudFront and SSM) and the static IAM keys from cluster workloads; the AWS keys left on the CI agents can only assume a role

---

7-2. OpenTelemetry End-to-end Distributed Tracing Pipeline (OTel + Tempo, 2026.07 ~ Present)

Problem

EFK logs and Prometheus metrics alone said little about how calls flow between microservices, where latency builds up, or how far an error spreads.

Approach

- **Tracing stack and auto-instrumentation**: an OpenTelemetry Collector and Grafana Tempo (S3), with the OTel Operator injecting the SDK without code changes
- **Metric extraction and storage split at different points**: RED metrics come from 100% of spans before sampling, keeping the aggregates exact, while storage keeps only tail-sampled traces — every error, every request over 500ms and 10% of normal requests

Results

- Metrics, traces and logs linked in one Grafana view, making cross-service bottlenecks visible, with a trace opening just its own request's logs by trace_id
- Connections pinned per collector replica had piled load onto one (metric series at 1,650 : 541); two-layer routing by trace ID now spreads it evenly across the two sampling instances at 4.09 : 5.18 spans/s

---

7-3. Zero-downtime Delivery on Argo Rollouts (Blue-Green → Canary, 2026.07 ~ Present)

Blue-Green promotion swapped the target group wholesale, so requests failed with 503 when old pods left before new ones registered.
A canary rollout now readies the new pod inside the same target group before the old one leaves, with deregistration delay (30s) < preStop (35s) < termination grace (60s), so backward-compatible patches to the public-facing main service ship with no downtime; a change that cannot run alongside the old version waits for a maintenance window.

7-4. Soft-Launch Operation (2026.07 ~ Present)

Results

- Soft-launch traffic runs on the Canary deploys and 3-signal observability, with HPA · Karpenter autoscaling in place
- Re-sized Pod requests and limits from two weeks of real soft-launch traffic, fixing requests so over-sized that the HPA could never scale out and cutting requested CPU by 48% and memory by 71%

7-5. SLI/SLO and Error Budgets (2026.08)

Built latency SLI recording rules and an SLI/SLO dashboard comparing error-budget burn, from request latency across every span before sampling, screening out targets that always pass ("p95 under 2s") against the measured distribution.
The game server answers HTTP 200 even when it throws, so not one of roughly 7.9M request spans was an error; an availability SLI that would read 100% through an outage waits until errors are returned as errors.

7-6. Edge Security for the Game CDN: AWS WAFv2 (2026.07 ~ 2026.09)

Put WAFv2 in front of the game's CloudFront distribution, which had faced the internet with no WAF ahead of launch. Four AWS managed rule groups observed live traffic in Count mode first, so no legitimate player was blocked, and a later rule blocks internal API paths outside the operators' IP allow-list.
A rate-based rule, which carrier CGNAT could turn against legitimate players, waits for data.

Project 8: Production Cutover to an EKS Auto Mode Cluster & a Site-to-Site VPN (WireGuard) Gateway

Contribution: data tier, IAM, CDN, VPN, observability and the cutover designed and run solo 2026.08 ~ 2026.11 (production cutover and old-cluster teardown done)

Problem

VPC peering with the publisher's account was required, so the data tier and workloads had to move onto a new VPC and EKS Auto Mode cluster built to the publisher's standard.

Approach

- **Compute layer moved onto EKS Auto Mode**: node lifecycle, the AWS resources Karpenter needs and the built-in addons are left to AWS, and the code keeps only NodePool declarations.
  NodePools and IngressClasses are applied by helmfile before ArgoCD, so a rebuild never leaves ArgoCD itself Pending
- **Site-to-site WireGuard VPN gateway**: more public ALBs would multiply hostnames, certificates and DNS records with every environment, so a tunnel keeps one domain and certificate, with the VPC route managed on the office router alone
- **The test-account EKS cluster replaced with ECS Fargate**: the whole cluster was running one proxy pod requesting 50m CPU and 128Mi of memory, so it moved to a Fargate service behind an internal ALB and the cluster was retired; EC2 at the same cost would lose rolling deploys
- **The CloudWatch alerting plane moved, and its blind spots closed**: alarms moved to a new stack at the cutover so the new data tier (Aurora, Valkey) never ran unwatched, notifier-Lambda errors go to an email path that bypasses Lambda, and the AWS Prometheus is watched by an on-premises dead-man's switch

Troubleshooting

- **Auto Mode ignoring the ALB group annotation**: the built-in controller ignores `group.name` and groups only through `IngressClassParams`, so 4 of the 5 service-group Ingresses would have had no ALB; each ALB group now declares its own IngressClass and IngressClassParams
- **CDN listener rules failing mid-cutover**: the Auto Mode managed policy allows only its own tag keys, so a custom ALB tag from the shared chart made listener-rule creation fail with 403; the tag was emptied and the copies already attached removed by hand

Results

- Terraform-managed compute resources cut 83% (102 → 17), with only the node-pool policy still managed directly, as NodePools
- 15 account-level assets (IAM, CDN, certificates, ECR) moved by state alone with zero deletions, so ARNs and addresses stayed put and no CI or manifest needed a change
- Production followed a proven review environment in a regular maintenance window: the schema moved in 53 seconds (about a minute in rehearsal) with row counts and checksums matching on all 36 tables, and the old cluster was torn down the same day
- Replacing the test-account cluster with ECS Fargate cut its monthly cost about 89%, from about $250 to about $28 (2026.10 Cost Explorer measurement → estimate at 2026.11 list price)

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Worked as a DevOps Engineer at a game/blockchain startup, leading the infrastructure side of a large-scale AWS → GCP migration.
Drove the move for cost optimization and GCP's global network, completing the full service migration within a scheduled game-maintenance window with minimal downtime. Kept dual source management (GitLab on-premises, GitHub for production) and upgraded the stalled bare-metal GitLab in place along the official path (about 12 hops).
Also built a pipeline consolidating MongoDB · CloudSQL · GA · Dune data into BigQuery with auto-refreshed Google Sheets, replacing analysts' 2-3 hours of daily manual collection.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Build-out

Contribution: Infra architecture design, migration strategy & execution lead 70% (remaining 30% = server-team application-side migration collaboration), CI/CD pipeline construction 100% 10 months (2023.03 ~ 2023.12)

Problem

AWS costs kept rising, driven largely by per-Pod billing for the always-on game workloads on Fargate.
The migration targeted cost reduction alongside the global load balancing the multi-region game service needed.

Approach

- **Three-phase sequential migration** - Phase 1 built and validated the dev environment on GCP, Phase 2 QA, Phase 3 production
- **Network and authentication** - Shared VPC (Host / Service Project split); GitHub Actions' GCP auth moved to Workload Identity Federation, replacing service-account keys with short-lived OIDC tokens.
  DNS was pre-validated by subdomain delegation (Hosting.kr → AWS Route53 → GCP Cloud DNS), leaving only the final NS swap for an announced maintenance window
- **Compute re-platforming** - AWS Fargate workloads moved onto GKE standard node pools; per-Pod billing loses to per-VM billing for always-on workloads, so GKE Autopilot — the same billing model — was ruled out
- **Storage minimum bypassed, WAF** - worked around Filestore's 2.5TB SSD minimum by self-hosting an NFS server on Compute Engine (pd-balanced 1TB disk), plus Cloud Armor region blocking and an IP allow-list WAF policy
- **IaC and GitOps pipeline** - codified GCP infrastructure with Terraform (100% excluding IAM), then after the migration built a GitLab CI + ArgoCD GitOps CI/CD pipeline standardizing per-environment config with Helm Charts

Troubleshooting

- AWS ALB sends traffic to every target once all of them fail health checks, where the GCP load balancer blocks it — so health-check failures AWS had been hiding turned into dropped traffic the moment the service moved.
  Rebuilt the health checks and backends around GKE attaching pods directly as NEG endpoints

- A 503 came from the GKE Ingress health check: the path set in BackendConfig returned nothing.
  Made that path always answer and aligned it with the ManagedCertificate and FrontendConfig settings, restoring external traffic
- After the move to Cloud CDN, the HTTP → HTTPS redirect was missing and game client downloads failed.
  Added a separate HTTP URL map that only redirects to HTTPS and pointed the port-80 forwarding rule at it

Results

- Cut cloud cost 50% through the AWS → GCP migration, the move from Fargate to GKE standard node pools, and an MSP reseller discount combined
- Completed the full migration in three phases: AWS kept serving while the same setup was built fresh on GCP, and only the final DNS cutover ran inside an announced maintenance window of about two hours, keeping downtime short
- Codified the GCP infrastructure as Terraform IaC, IAM aside, with every deployment traceable through Git commits
- Eliminated GCP service-account key management overhead and key-leak risk for GitHub Actions by adopting Workload Identity Federation

---

Project 2: Game Data Collection Automation System

Contribution 100% (Python script development & operations) 3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, costing analysts 2-3 hours daily and causing frequent data gaps from human error.

Approach

- **Multi-source ingestion into BigQuery** - MongoDB via a Dataflow Flex Template ETL, CloudSQL via BigQuery's direct connection, Google Analytics · Dune via their own APIs (Dune with a separate API key), all normalized into a common schema
- **Auto-refreshed analyst-facing Google Sheets** - Python Cloud Functions push BigQuery query results into Google Sheets via the Sheets API, triggered by Cloud Scheduler on Daily (previous-day) and Monthly (previous-month) cadences
- **Full infrastructure as Terraform IaC** - BigQuery datasets, Cloud Functions, schedulers, Cloud Storage, Artifact Registry, IAM and service accounts all managed as code

Troubleshooting

- The CloudSQL → BigQuery federated query failed on a region mismatch and missing connection-SA permissions; fixed by creating the connection in the same region and granting that SA the Cloud SQL Client role
- Dune API's async execute → poll → fetch flow and rate limits caused missing results and timeouts; status polling with backoff now guarantees retrieval, and the API key moved into Secret Manager
- Daily re-runs loaded duplicate records into BigQuery. Applied MERGE and ROW_NUMBER()-based reconciliation logic in the dedup Cloud Function to make the load idempotent
- Writing BigQuery results through the Google Sheets API failed on the per-minute quota (429) and on results too large for one write; results now go 1,000 rows at a time into per-date sheets, with pauses between calls and exponential-backoff retries on quota errors

Results

- Replaced analysts' 2-3 hours of daily manual collection with automated loading, ending the gaps from manual-entry mistakes
- With the whole stack in Terraform, the same pipeline was rebuilt quickly in three other GCP projects, every change traceable in Git

---

Project 3: GPU Infrastructure Standardization & Scale-out for AI Workloads (Windows → Linux · Automated Setup · On-prem / GCP / External IDC)

Contribution 100% (Infra design, build & operations) 6 months (2024.02 ~ 2024.07)

Problem

The in-house GPU server ran Windows, so every Stable Diffusion-family image-generation run meant hand-matching driver, CUDA and framework versions — hard to reproduce, and slow to get to work despite the hardware.

Approach

- **Windows → Linux standardization** - GPU servers moved to Ubuntu 22.04; one setup script covers the driver stack (nvidia-driver-550 · CUDA 11.8 · cuDNN 8.6), GPU checks, the shared NFS model store and a Stable Diffusion WebUI per GPU, plus kohya_ss and ComfyUI
- **Cloud scale-out on GCP** - A100 40GB and L4 VMs in Terraform, reshaped with demand up to four A100s and eight L4s, joined to the migration's Shared VPC subnet after zone availability checks and quota increases
- **20 more servers at an external IDC, card tier split by workload** - 20 GPU servers from the same script; training runs on GCP A100s for their VRAM, and bulk inference, which scales with GPU count, on consumer RTX 40-series servers at the IDC

Results

- Replaced Windows hand-configuration with Linux and a setup script, the standard build for every in-house, GCP and IDC GPU server
- GCP GPU VMs managed in Terraform, so every reshaping and rebuild is tracked in code — folded into the existing GCP IaC repository
- Scaled GPU capacity from one in-house box to 10 in-house, GCP A100/L4 VMs and 20 IDC servers, card tiers split by workload

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Built PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients and provided technical support.
Designed and built private cloud infrastructure for financial and public sector clients, and led training for partner engineers.

Project 1: Customized PaaS Solution for Clients

Contribution 100% (Infra design & Kubernetes cluster build) 11 months (2022.04 ~ 2023.03)

Problem

Each client had different environment and server spec requirements, making standardized solutions insufficient.

Approach

- **HA cluster design** – Kubernetes clusters designed for high availability per client requirements, with VM management built on OpenStack
- **Unified storage** – block, file and object storage provided together through Ceph

Troubleshooting

- **Planned shutdown and restart** – client power work meant cutting power, so zero downtime was impossible and the goal became stopping without corrupting data and coming back cleanly. VMs stop first, then the DB and message queue, with storage last, and restart runs in reverse.
  The DB cluster (MariaDB Galera) is shrunk one node at a time and restarted from the node holding the latest data, and storage (Ceph) is set beforehand so a powered-off node is not treated as a failure that starts data recovery. Restart failures the field could not resolve (a message queue node not rejoining the cluster, volume creation failing) were recovered as second-line support

Results

- Built and delivered the HA-configured PaaS solution to 5 clients and ran it stably
- Automated server provisioning and configuration with Ansible — the same playbooks for every client, and shorter deployments
- Handed the shutdown and restart runbook for power work to field engineers, with the fixes for restart failures written into the same runbook

Project 2: Kubernetes Operations Training Program for DP

Contribution 100% (Training design & delivery) 6 months (2022.06 ~ 2022.11)

Problem

DP's field engineers lacked Kubernetes and container experience, so running the PaaS on their own after adoption was expected to be hard.

Approach

- **Step-by-step curriculum** – Kubernetes basics through cluster operations and troubleshooting, taught practice-first in lab environments
- **Parallel PaaS-stack training** – OpenStack and Ceph storage operations taught alongside, covering the full PaaS stack

Results

- Monthly 4-hour sessions for six months (about 10 attendees) and an operations guide let DP's field engineers run Kubernetes on their own with fewer inquiries; the materials were reused for later clients